

Regression Vs. Classification

Regression is a problem of predicting a continuous numeric value (e.g., a price, a temperature, a score) based on input data.

Regression



What will be the temperature tomorrow?

84°



Fahrenheit

Classification



Will it be hot or cold tomorrow?

COLD

HOT



Fahrenheit

Computer Vision Tasks

Classification



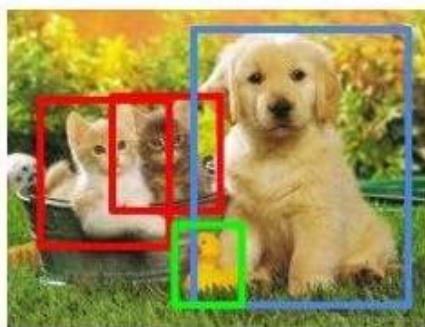
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**



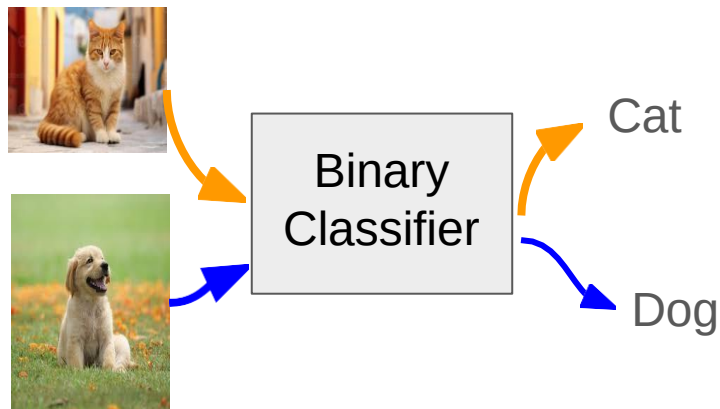
CAT, DOG, DUCK

Single object

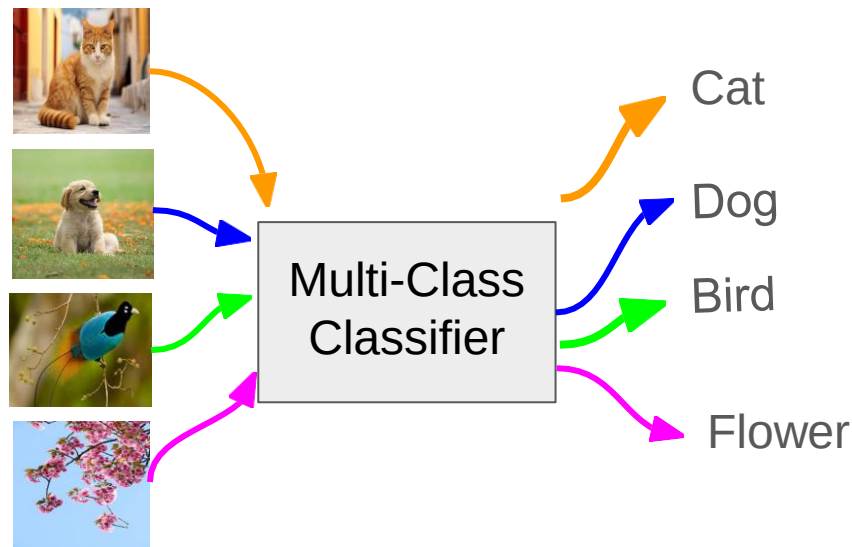
Multiple objects

Classification

- Assigning objects to some pre-existing classes / categories / labels / groups.



Number of classes = 2



Number of classes = 4

Fully Connected Neural Network (FCNN)

FCNN consists of a series of fully connected layers

It is also known as multi-layer perceptron (MLP).

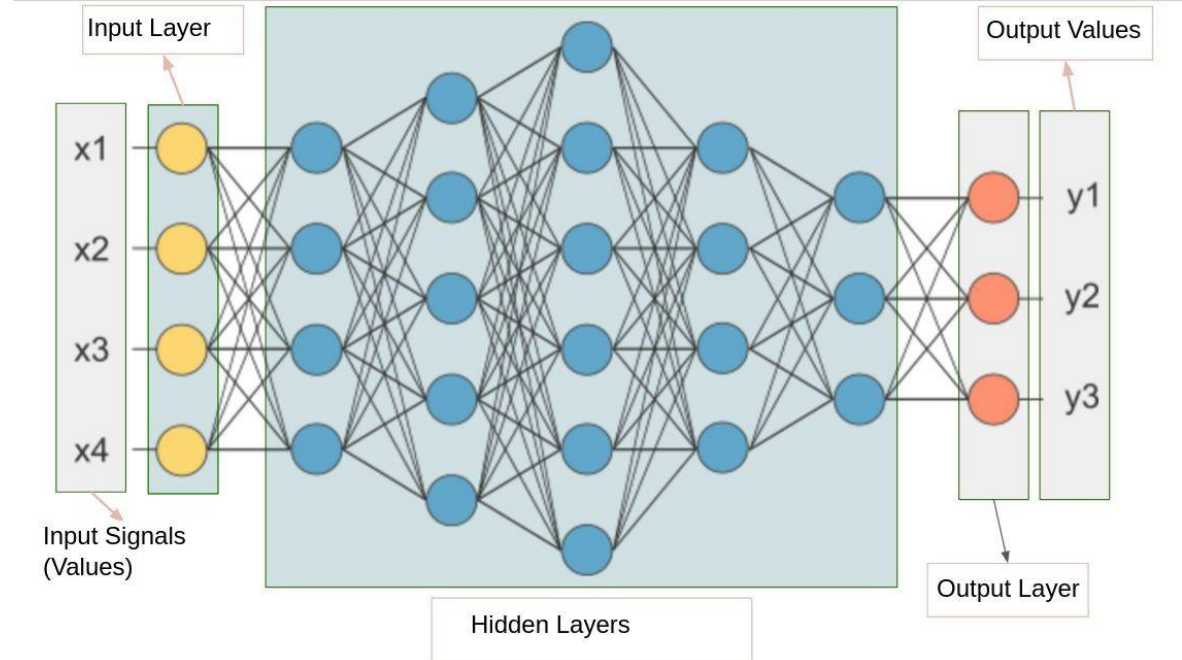


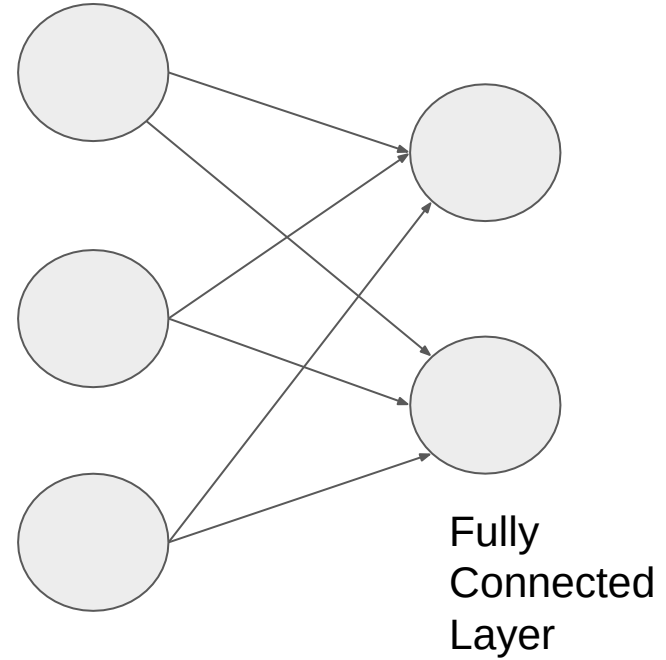
Image Source: Google Search Engine

Fully Connected Layer

A fully connected layer connects every neuron in one layer to every neuron in its previous layer.

An image need to be turned into a vector before feeding into an fully connected layer.

Flatten() is used in
Tensorflow.keras



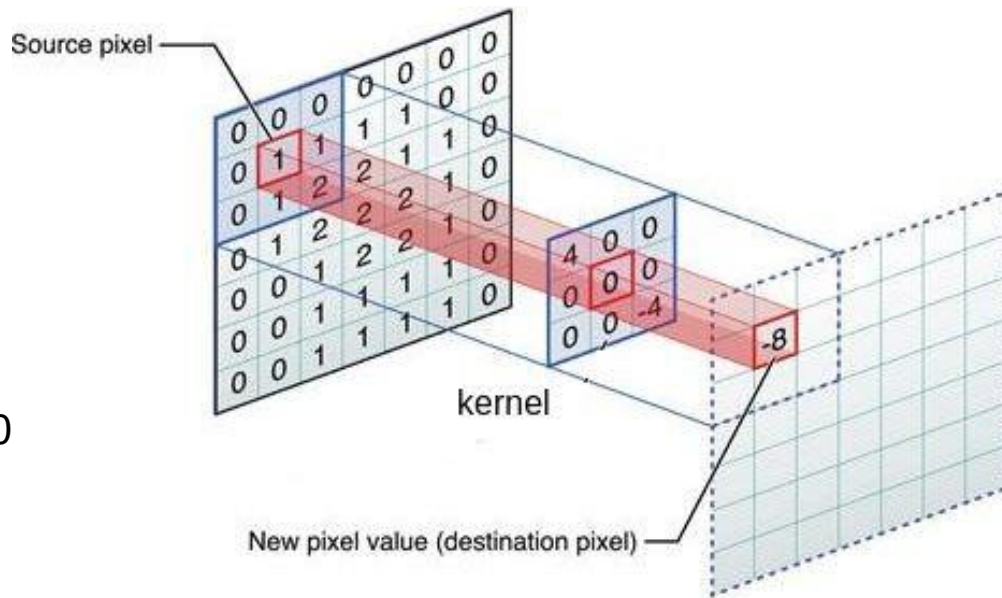
Convolution for Single-Channel Input & Single-Channel Output

Convolution layer adds each element of a feature map to its local neighbors, weighted by the kernel.

Source pixel: 1

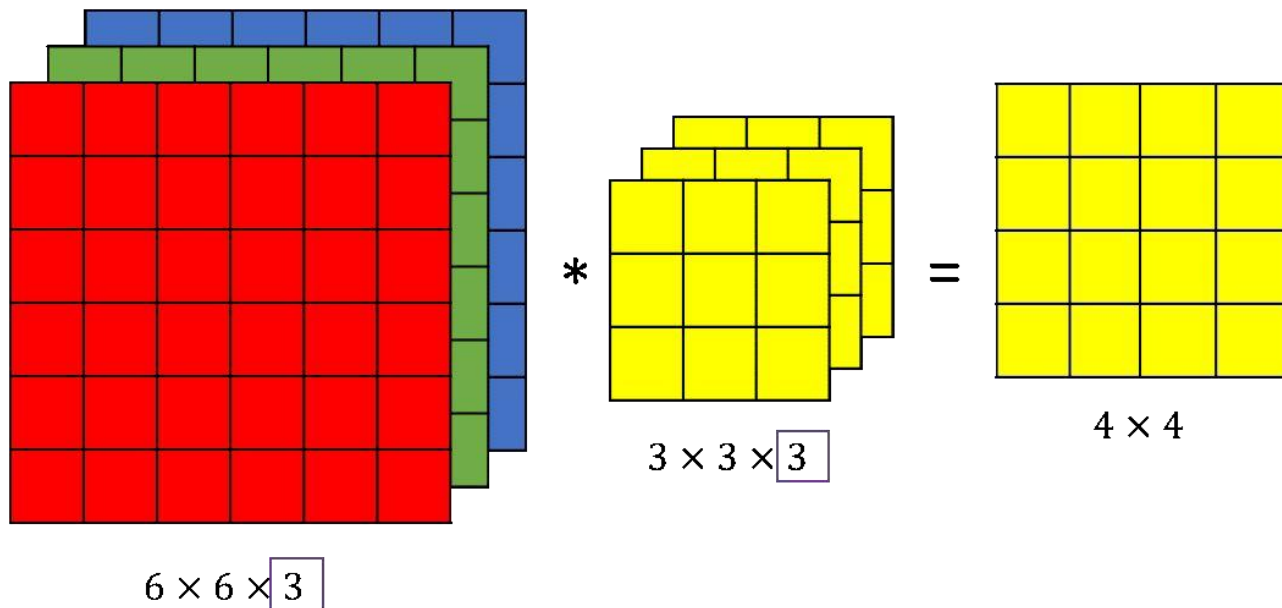
Destination pixel: $4 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0$
 $+ 1 \times 0 + 2 \times 0 + 0 \times 0 + 1 \times 0 + (-4) \times 2 = -8$

Total number of multiplication for one pixel = kernel_height x kernel_width



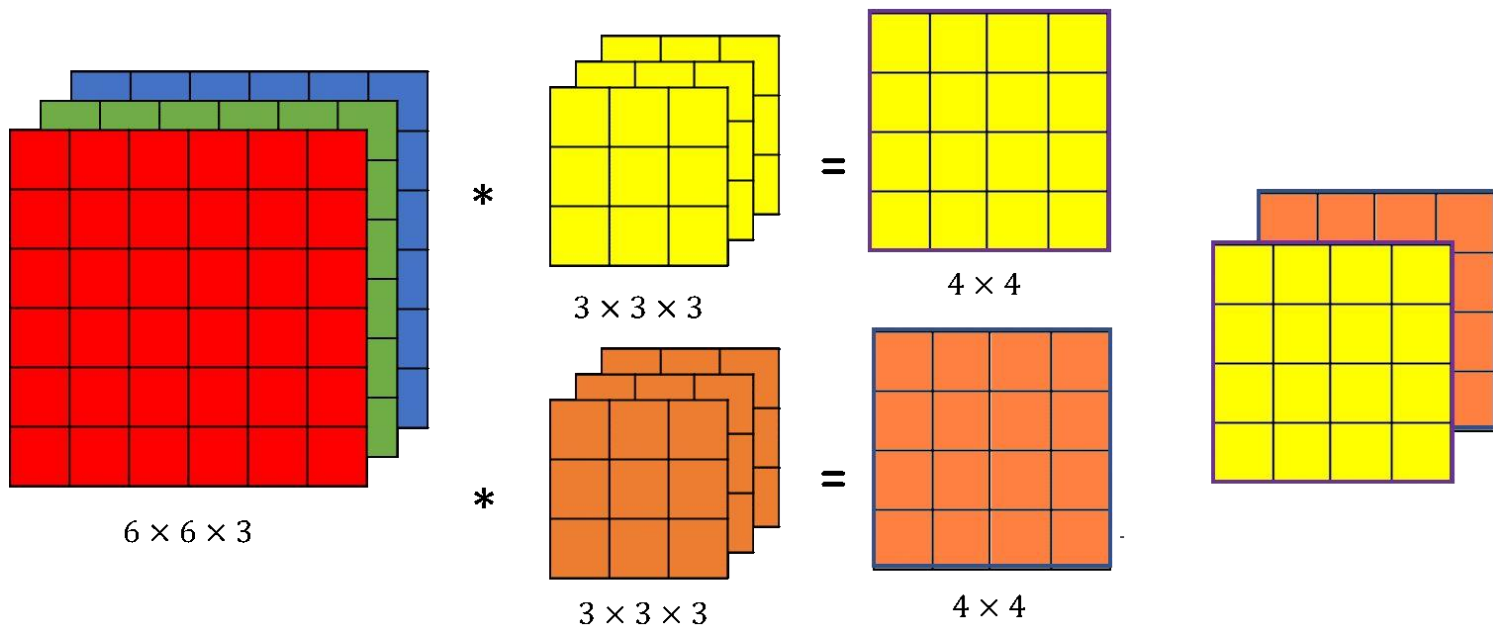
Convolution for Multi-Channels Input & Single-Channel Output

- Number of Kernel's Channels = Number of Input Feature Map's Channels



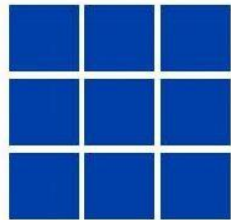
Convolution for Multi-Channels Input & Multi-Channels Output

- Number of Kernels = Number of Output Feature Map's Channels



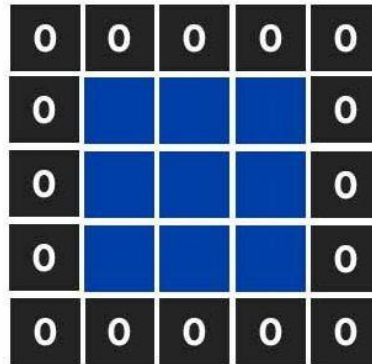
Padding

Padding preserves the spatial size of the input so that the output after convolution remains the same size or desired size.

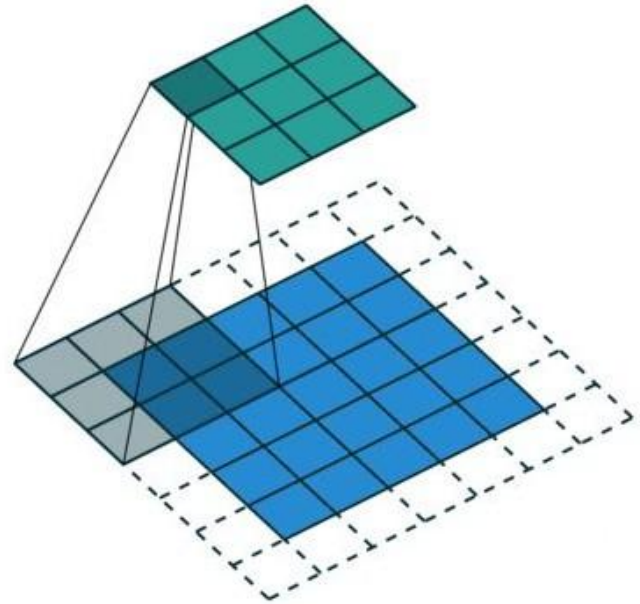


Input Image

Applying padding
of 1 on 3X3



Padded Image



Steps of Traditional Machine Learning

1. Data Preparation:
 - a. Data Collection
 - b. Data Splitting
2. Data pre-processing
 - a. Data Cleaning
 - b. Data Transformation
3. Feature Extraction:
 - a. Identify Relevant Features
 - b. Extract and Transform Features
4. Training Classifier:
 - a. Model selection
 - b. Searching for optimum parameters
5. Evaluation
 - a. Estimate metrics using test set

End-to-End Learning

End-to-end learning or end-to-end training, is a machine learning model that:

- learns directly from raw input data to produce a desired output, without the need for manual feature engineering or pre-processing steps.
- gets popularity in deep learning.
- bypasses few steps of traditional learning such as data pre-processing, feature extraction.

Kernel

- In computer vision, kernel is a (usually) small matrix of numbers that is used in image convolutions.
- Different sized kernels containing different patterns of numbers produce different results under convolution.
- Generally, small odd numbers such as 3, 5, or 7 are used as the size of a kernel.
- Popular human-decided kernels:
 - Gaussian Blur Kernel
 - Laplacian Kernel
 - Prewitt Kernel
 - Sobel Kernel

Feature Map

A feature map:

- is a 2D matrix in a hidden layer
- represents the output of a convolutional layer
- captures specific features or patterns in the input data.

In a CNN:

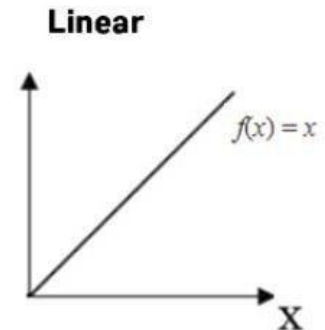
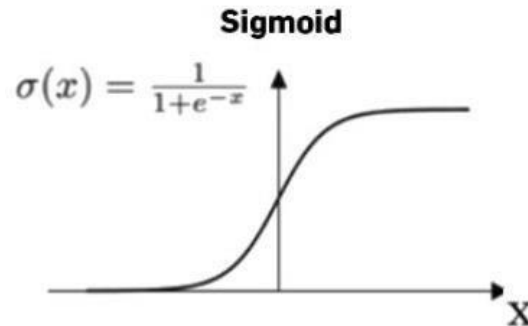
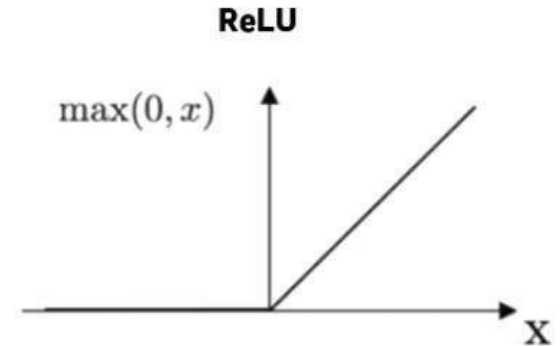
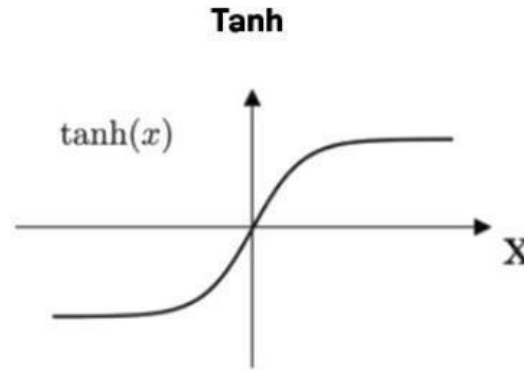
- Instead of human-decided kernels/filters, different automatically learned kernels/filters are used to generate different feature maps

Activation Function

- An activation function is a mathematical equation that determines how much data should be passed from a neuron to the next neuron.
- It is the function that is applied on the weighted sum of the input of the neuron, i.e., it is $g(u)$.
- Generally, it is a nonlinear function for a neuron in the hidden layer and linear/non-linear function for neurons in the output layer.
- A linear activation function, also known as "no activation" or "identity function," is directly proportional to the input.
- Popular non-linear activation functions are:
 - Sigmoid, Tanh, ReLU, ELU, Softmax

Activation Functions

- Linear activation function in the output layer generally used for regression problem.
- Depending on the range of output values, we need to choose activation function.



Some Popular Activation Functions

- **Sigmoid**
 - a. Also known as the logistic activation function
 - b. Often used for models that predict probability as an output.
 - c. Its curve looks like an S-shape and exists between 0 and 1.
 - d. Suffers from saturating gradients problem.
- **Hyperbolic tangent (Tanh)**
 - a. Has stronger gradients than the sigmoid function, and its output ranges from -1 to +1.
 - b. Helps the learning algorithm converge faster.
- **Rectified linear unit (ReLU)**
 - a. A non-saturating function, meaning it doesn't become flat at the extremes of the input range.
 - b. It is faster than sigmoid and tanh.
- **Softmax**
 - a. Converts vectors of real numbers into a probability distribution.
 - b. Each output value represents the probability that the input belongs to a specific class.

Activation Functions for Classification

In output layer, generally:

- Sigmoid is used for binary classification
- Softmax is used for multi-class classification

Both generate values in the range of 0-1.

Summation of softmax values is 1.

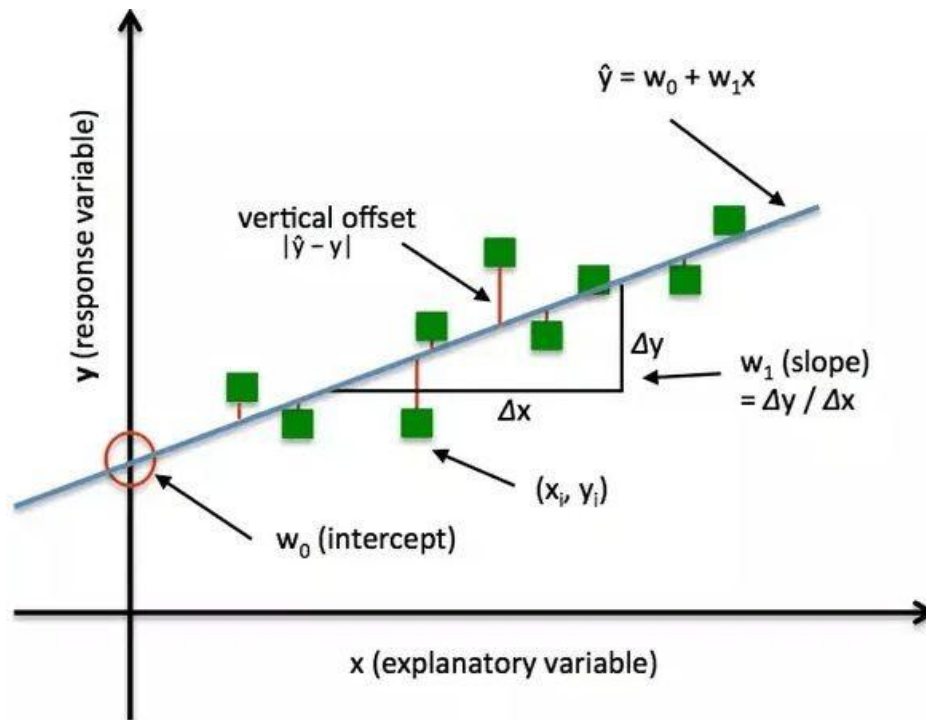
Summation of sigmoid values in a classifier does not need to be 1.

$$\text{sigmoid}, y_i = \frac{e^{x_i}}{1 + e^{x_i}}$$

$$\text{softmax}, y_i = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}$$

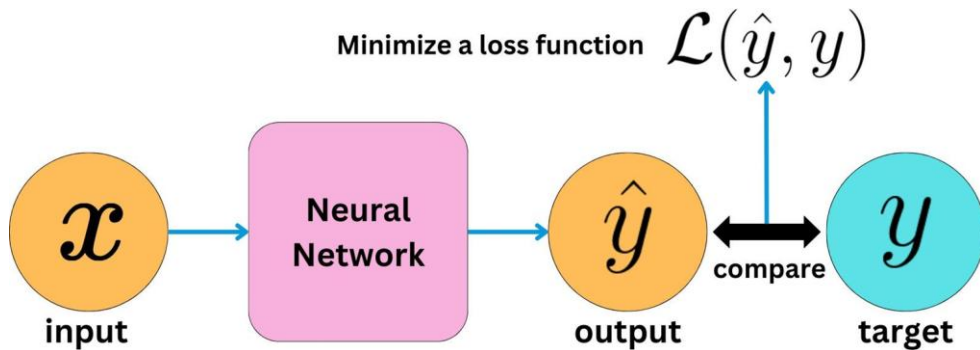
Loss

- Error is a measurement of the difference between the predicted output of the model and the actual output.
- Loss is the weighted error.
- Different types of loss:
 - Absolute loss: $|\hat{y} - y|$
 - Squared loss: $(\hat{y} - y)^2$
 - Log loss: $-y \cdot \log(\hat{y})$



Loss Function

- A loss function is a function which estimates the total loss of all data samples.
- It is used to find optimum values of parameters of a neural network during training.
- It is a function of parameters.
- Different ways of writing:
 - $J(\omega)$; $L(\theta|x)$; $\ell(\omega)$; $\mathcal{L}(\hat{y} - y)$;



Different Loss Functions

Different loss functions penalizes wrong predictions differently

Popular losses:

- Regression:
 - Mean-Squared Error (MSE)
 - Mean Absolute Error (MAE)
- Classification:
 - Binary Cross-Entropy
 - Categorical Cross-Entropy

L1 & L2 Loss

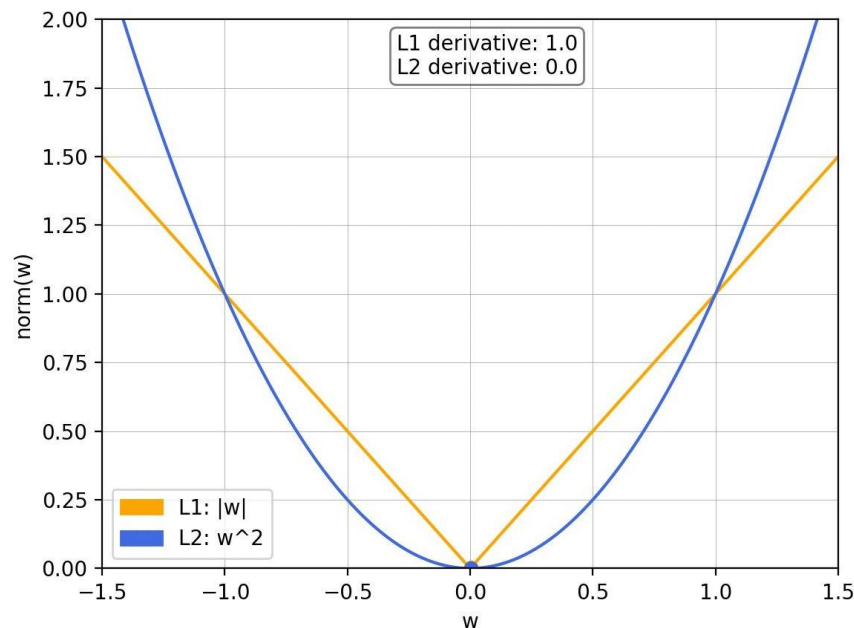
- Absolute Loss is also known as L1 loss
- Squared Error Loss is also known as L2 loss

$$L_1 = \sum_{i=1}^n |y_{gt} - y_{pred}|$$

$$L_2 = \sum_{i=1}^n (y_{gt} - y_{pred})^2$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_{gt} - y_{pred}|$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_{gt} - y_{pred})^2$$



Training Neural Network

During training we try to find the parameter set for which loss is minimum. Starting at random values, we update parameters by gradient descent algorithm

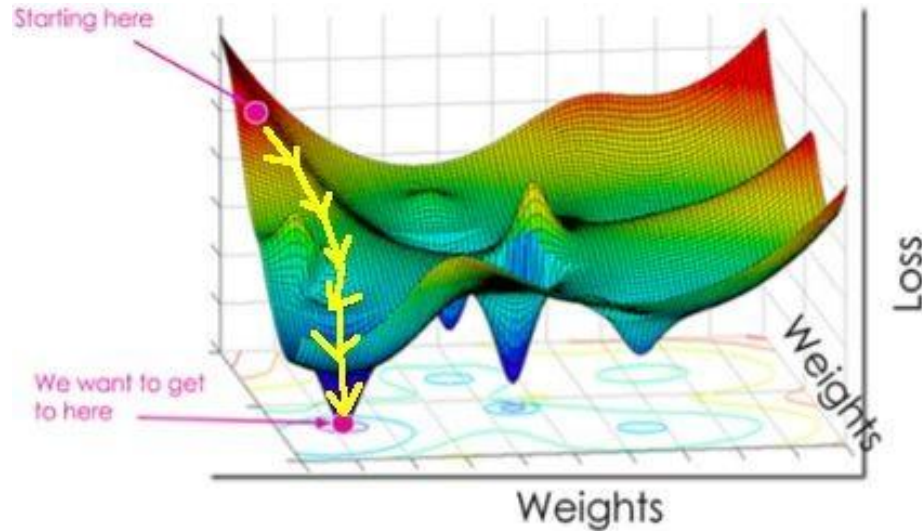
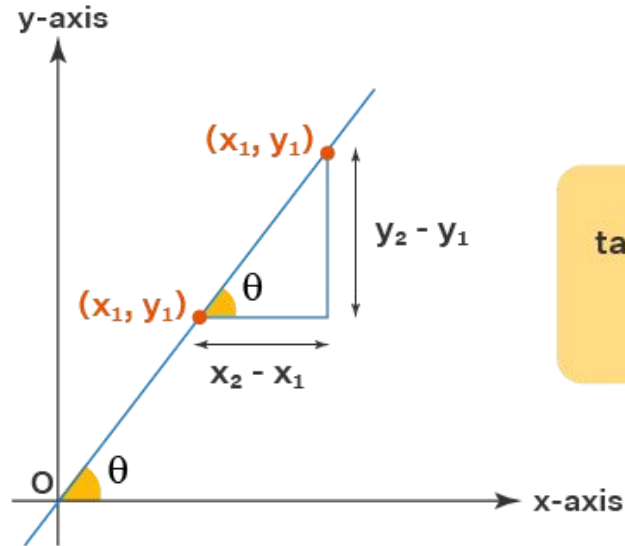


Image Source: Google Search Engine

Gradient

- Gradient (GD) means the change in the value of a quantity with change in a given variable
- the sign of a GD represents the direction of greatest change in a scalar function

Gradient of a Line



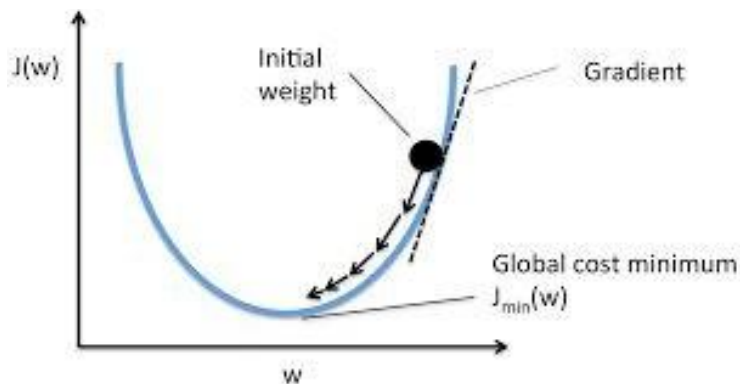
$$\tan \theta = \frac{y_2 - y_1}{x_2 - x_1}$$
$$m = \tan \theta$$

Gradient Descent Algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J$$

}

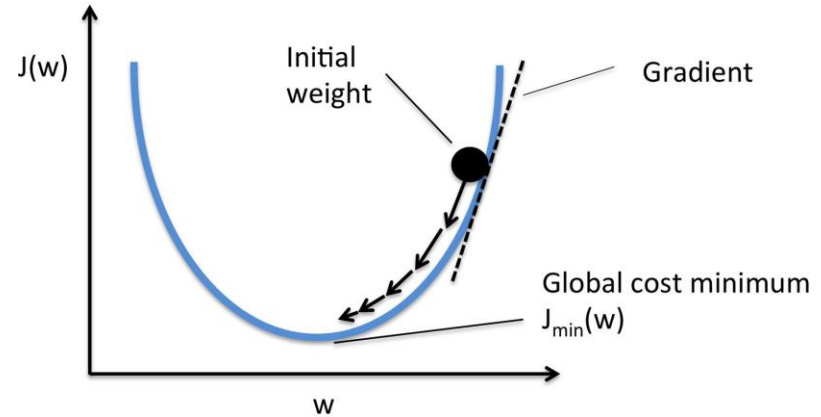


Gradient Descent Algorithm

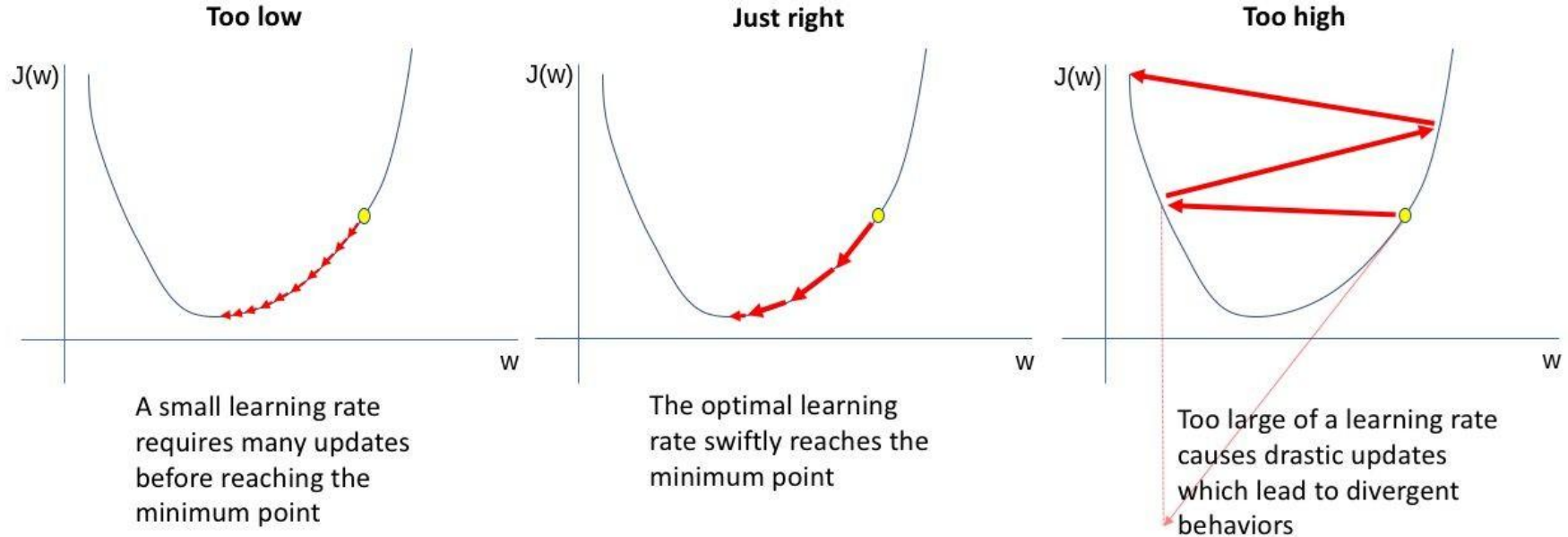
- Initialize parameters randomly
- Estimate outputs for inputs in a forward pass
- Estimate loss
- Estimate gradient of the loss function with respect to all parameters
- Move parameters in the opposite direction of gradient

$$w = w - \alpha \frac{d}{dw} J(w)$$

- Repeat these steps until loss is minimum



Effect of Learning Rate, α



A small learning rate requires many updates before reaching the minimum point

The optimal learning rate swiftly reaches the minimum point

Too large of a learning rate causes drastic updates which lead to divergent behaviors

$$w = w - \alpha \frac{d}{dw} J(w)$$

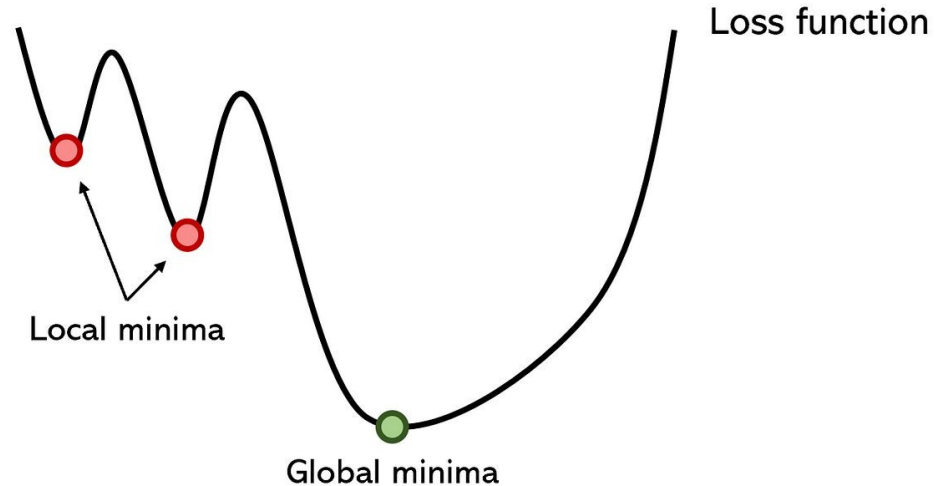
Local Vs. Global Minima

- **Local Minimum**

A point where a function's value for a variable parameter is smaller than nearby points, but possibly larger than at a distant point.

- **Global Minimum**

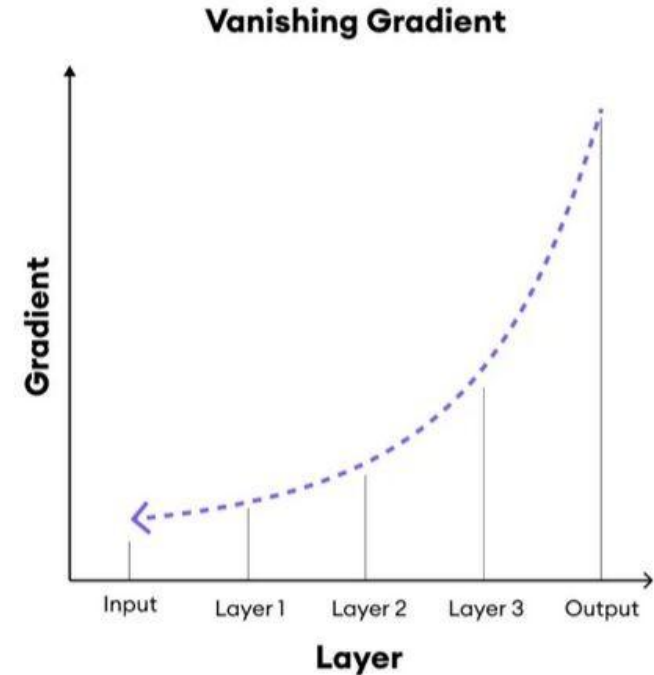
A point where a function's value for a variable parameter is smaller than at all other feasible points



Our target is to reach the global minima, but unfortunately we end up at a local minima.

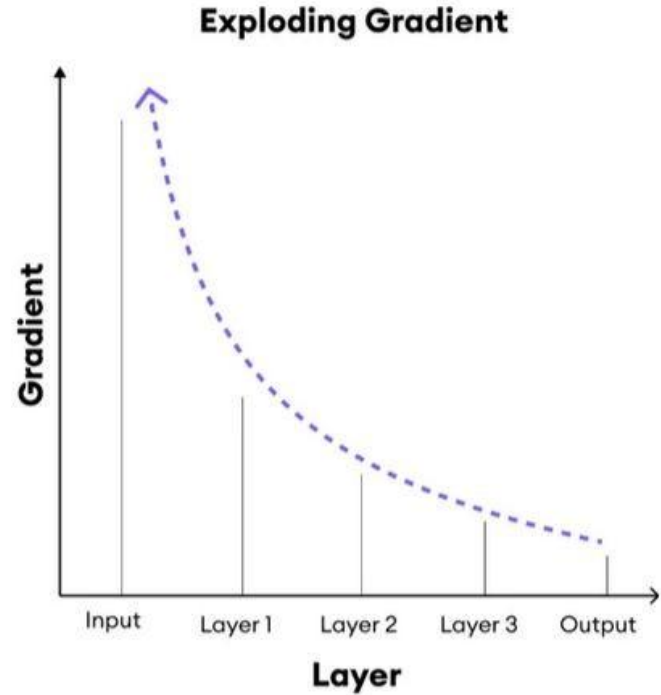
Vanishing Gradient

- gradients often get smaller and smaller and approach zero
- therefore, the weights of the initial or lower layers nearly unchanged
- as a result, the gradient descent never converges to the optimum
- model learns very slowly and perhaps the training stagnates at a very early stage just after a few iterations



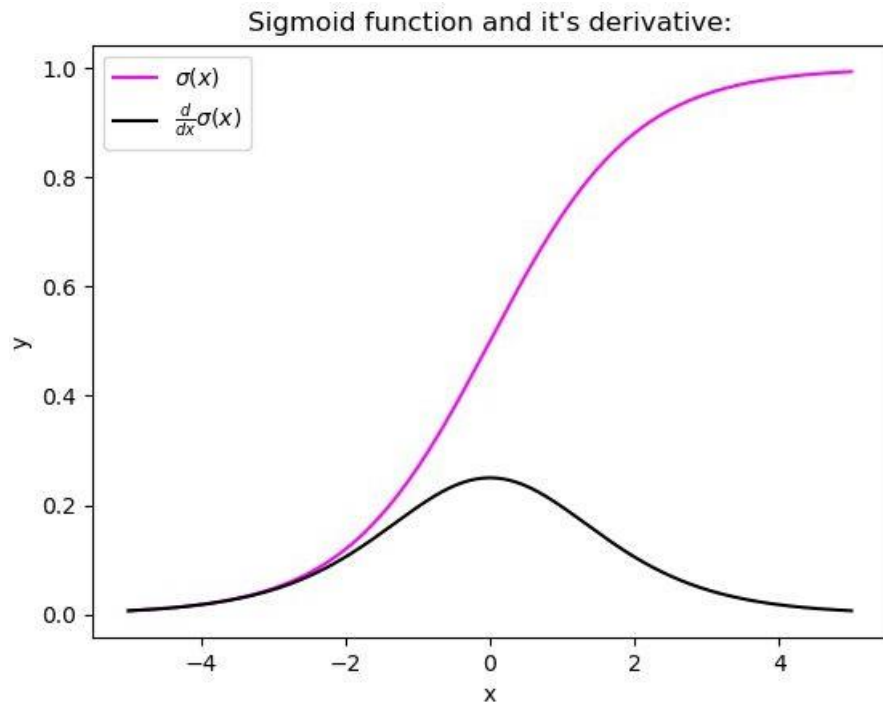
Exploding Gradient

- gradients keep on getting larger and larger
- it causes very large weight updates
- an exponential growth in the model parameters
- model weights may become large during training
- it causes the gradient descent to diverge
- model experiences avalanche learning



Ways to Handle GD Vanishing and Explosion

- Proper weight initialization
 - Glorot / He / He initialization
- Non-saturating Activation Functions
 - ReLU
- Batch normalization
 - Zero centering and normalizing inputs
- Gradient clipping to mitigate gradient explosion
 - clip all the partial derivatives of the loss with respect to each trainable parameter between -1.0 and 1.0

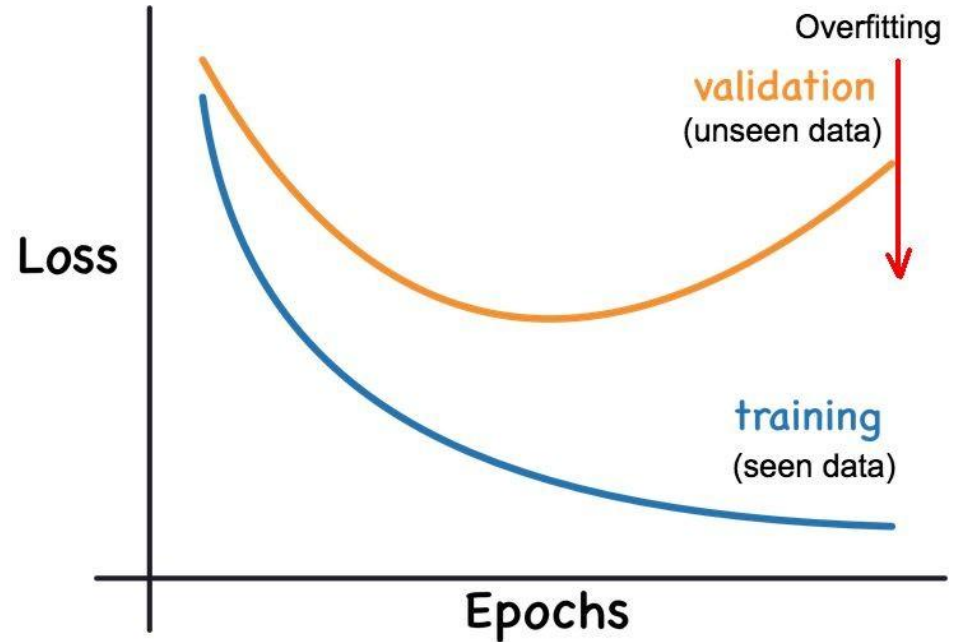


Overfitting

Overfitting occurs when:

- the model is so closely aligned to the training data that it does not know how to respond to new data.
- the model memorizes training data

Memorizing is not learning



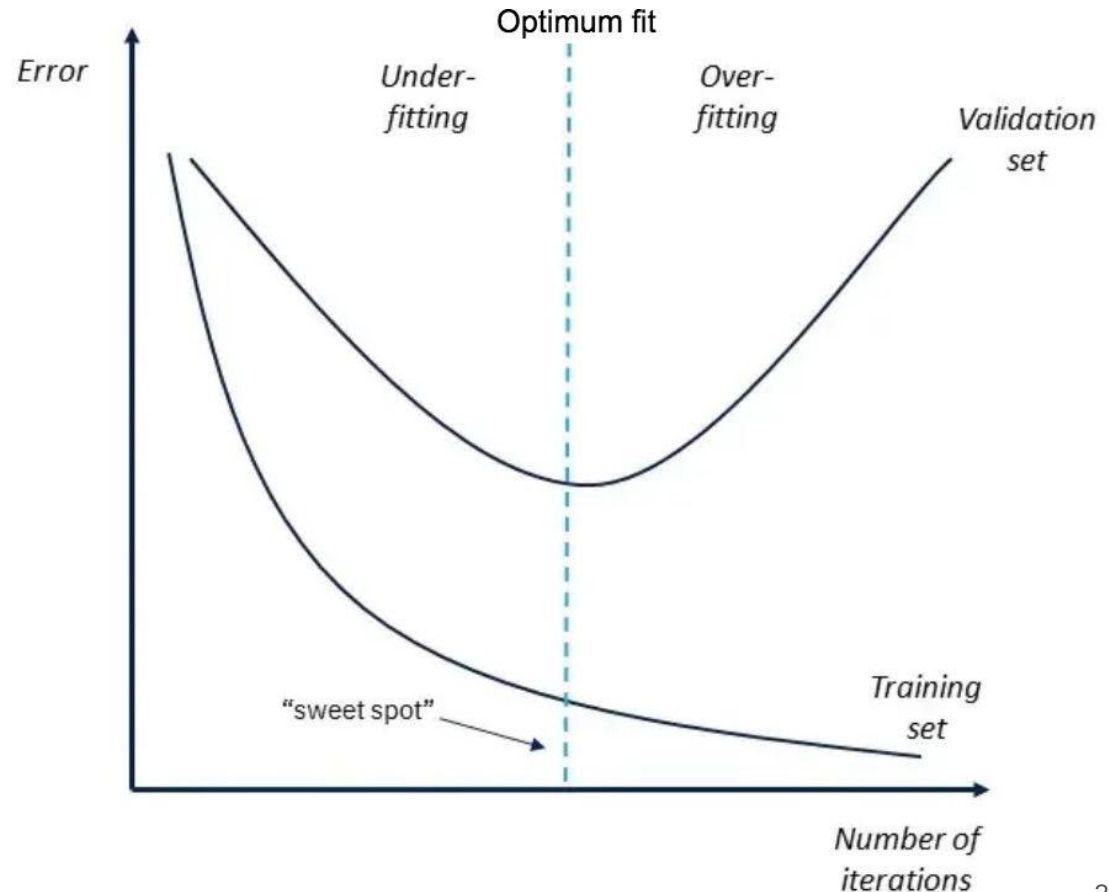
Reasons behind Overfitting

Overfitting can happen because:

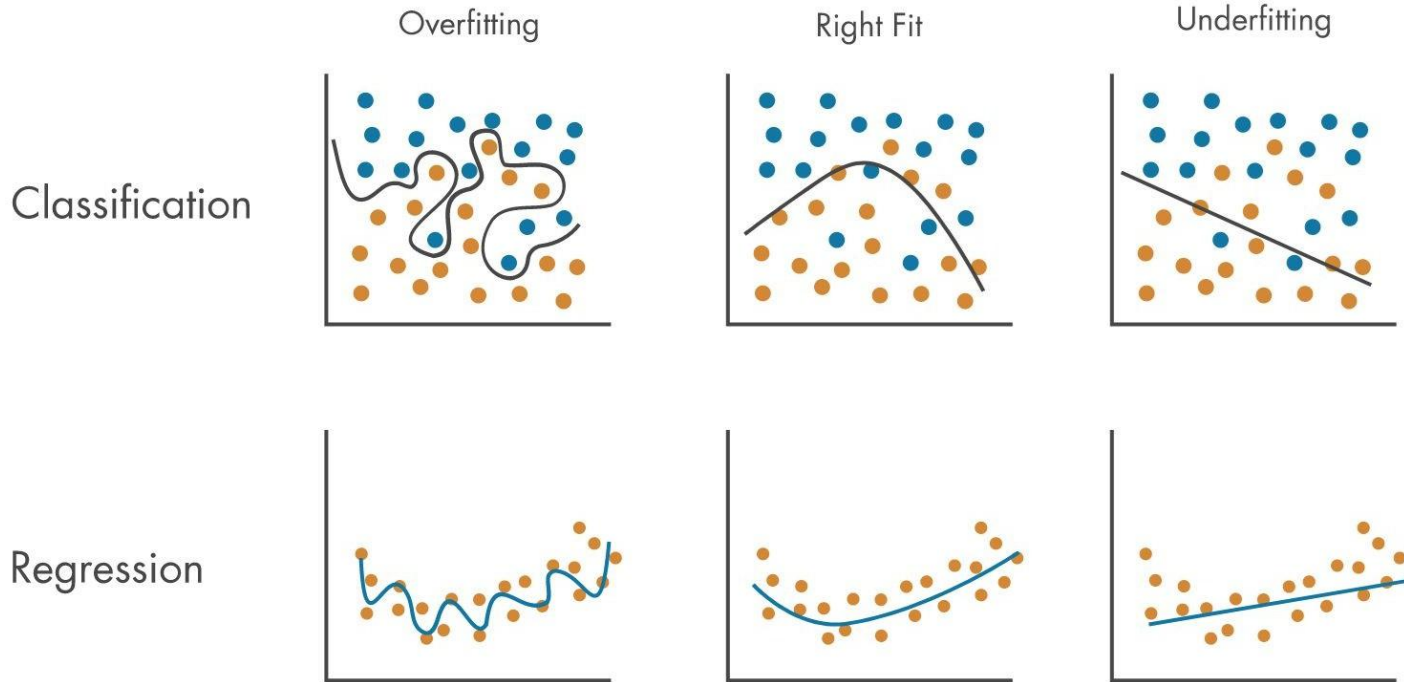
- Model is too complex; it memorizes very subtle patterns in the training data that don't generalize well.
 - Model has too many parameters
- The training data size is too small for the model complexity and/or contains large amounts of irrelevant information.
- During training model sees a small dataset for too many times i.e., too many epochs were used during training.

Underfitting

- Underfitting is the opposite concept of overfitting
- When the prediction error on both the training and validation dataset is high, and the difference between them is very minimal, the model is said to have under fitted.



Overfitting - Underfitting in Classification and Regression



Overfitting vs Underfitting

When only looking at the computed error of a model for the training data, overfitting is harder to detect than underfitting.

Error	Overfitting	Right Fit	Underfitting
Training	Low	Low	High
Test	High	Low	High

More details:

<https://www.mathworks.com/discovery/overfitting.html>

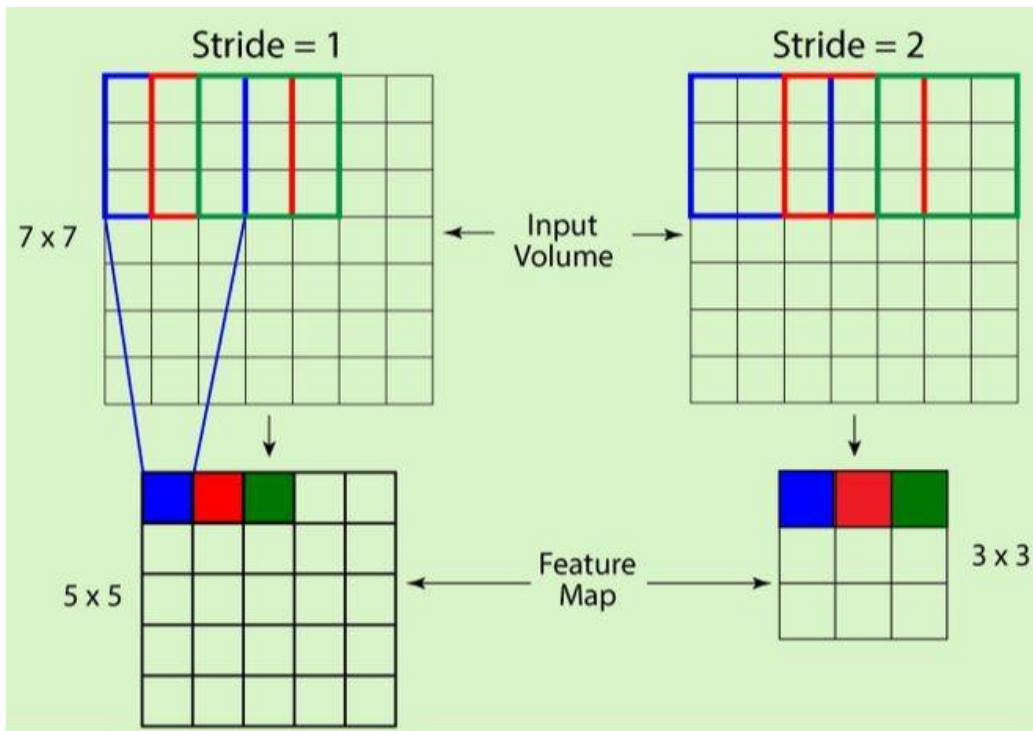
Downsampling Layer

A *downsampling layer* reduces the dimensionality of the features at cost of a some loss in information.

- Convolution layer with larger strides and no padding
 - It is a learnable operation, whereas pooling is a fixed operation
 - Some research works showed better performance
- Different pooling layers
 - Max pooling
 - Average pooling
 - Global average pooling

Downsampling by Convolutional Layer

Without padding, setting appropriate stride value, we can downsample feature map by convolution.



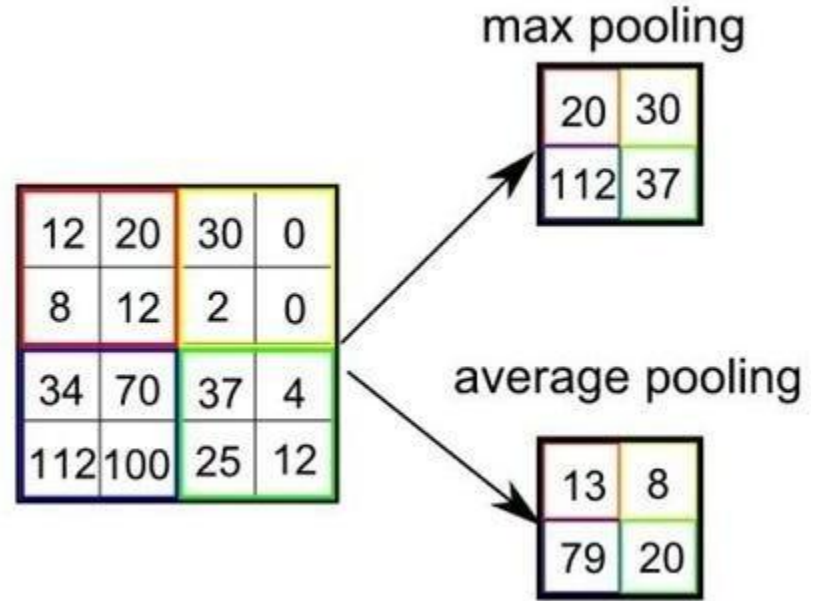
Pooling Layer

It reduces the spatial dimensions of feature maps, thereby:

- decreases computational cost
- makes the network more robust to variations in the input

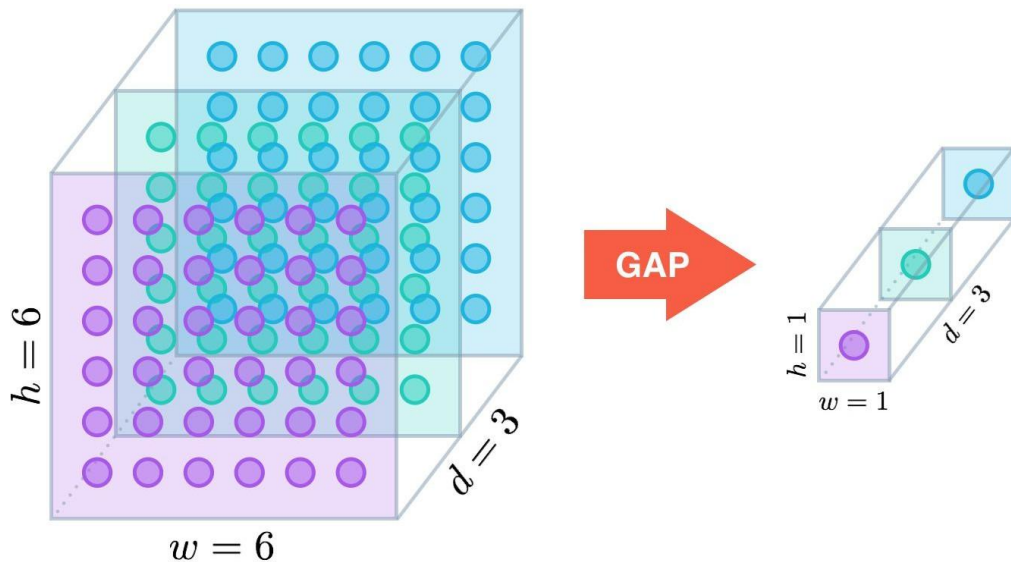
Different Types of Pooling Layers:

- Max Pooling
- Average Pooling
- Global Pooling
- Stochastic Pooling (not very common)

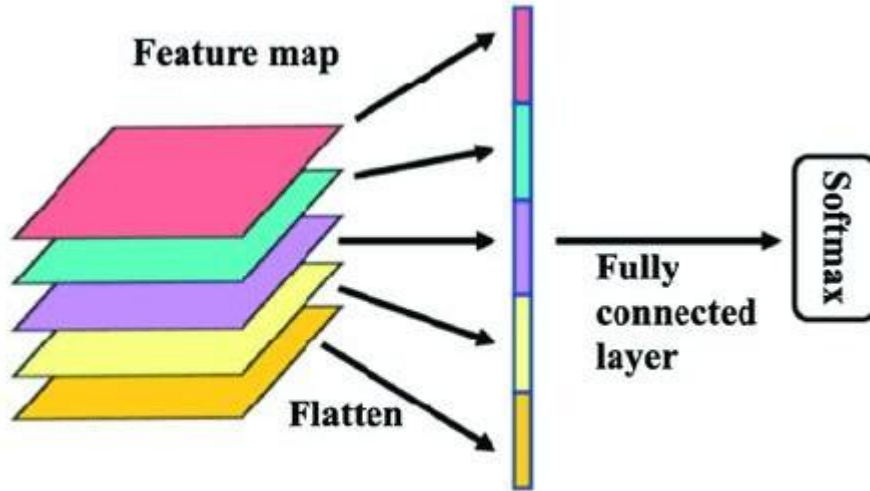


Global Pooling

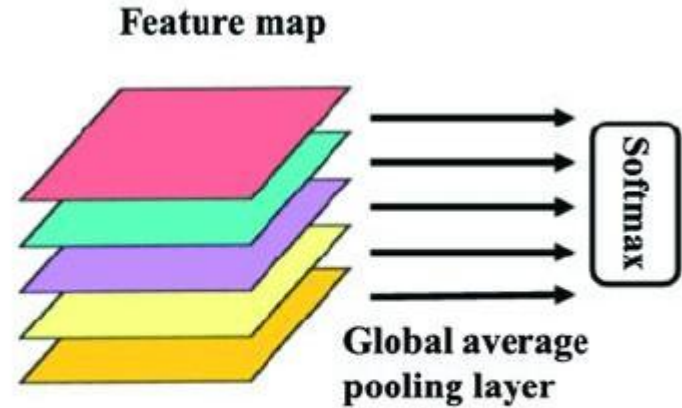
- It reduces the spatial dimensions of feature maps to a single value per channel.



Usage of GAP



(a) Fully connected layer



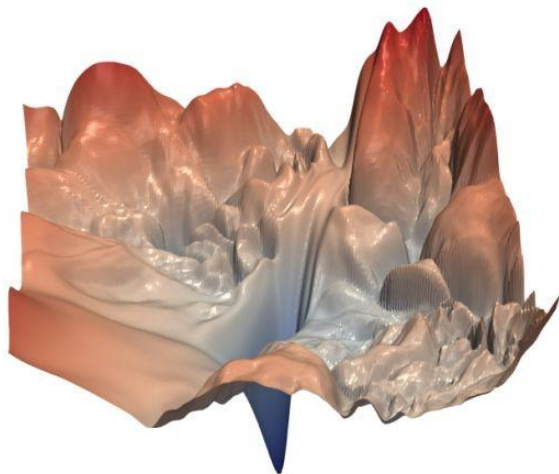
(b) Global average pooling layer

Skip Connections

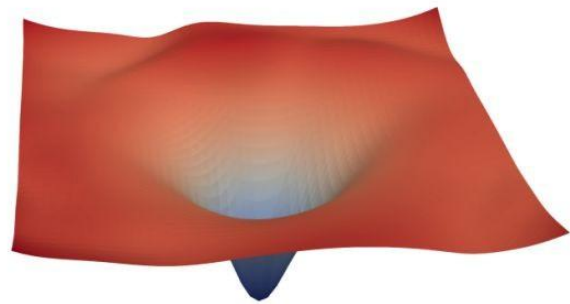
Skip connections allow CNNs to bypass some layers and connect directly to deeper or shallower ones.

Skip connections lead to:

- faster convergence
- vanishing gradient problem prevention
- loss of information prevention



(a) without skip connections



(b) with skip connections

How to Avoid Overfitting

- Model Validation
 - The error of an overfitted model is low when computed for the training data. We need to test performance of our model on a separate data set (i.e., validation data set) before using real-time application data.
- Regularization
 - Regularization applies either a penalty for complexity or roughness to the model. By introducing additional information into the model, regularization algorithms can deal with multicollinearity and redundant predictors by making the model more parsimonious and accurate.
 - <https://www.youtube.com/watch?v=wNdYpD2YhSk&t=331s>

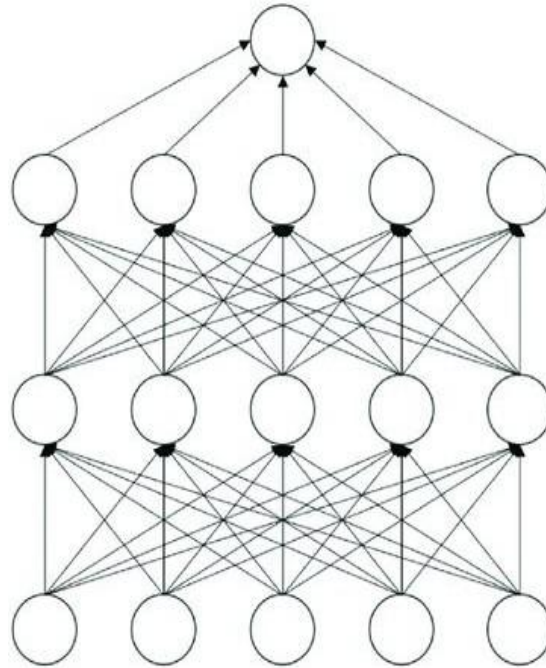
How to Avoid Overfitting

- Data Augmentation
 - When data availability is limited, data augmentation is a method to artificially expand the data points of the training data set by adding randomized versions of the existing data to the data set.
- Synthetic Data
 - Synthetic data generated by generative model can bring variability in available data, increase size of training dataset and reduce overfitting issue to some extent.
- Data Clean Up
 - Data noisiness contributes to overfitting. One common approach to reduce undesired data points is to remove outliers from the data.

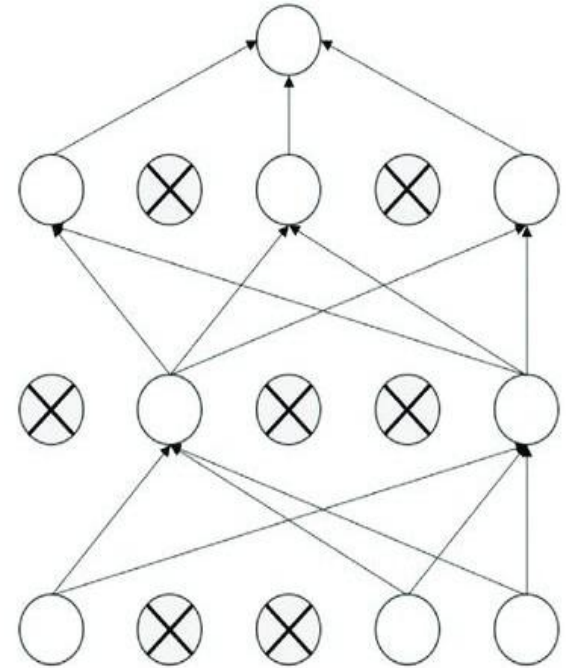
Dropout Layer

In dropout layer:

- individual nodes are excluded in various training runs
- prevents overfitting



Standard Neural Net



After applying dropout

Dropout Layer

- It activates during training phase only, not in validation and testing phase
- Forward Pass:
 - A certain percentage of neurons in the dropout layer are randomly picked to be "dropped" (deactivated).
 - The output of the dropped neurons is set to zero, effectively removing them from the network's calculations for that specific iteration.
- Backward Pass:
 - The gradients for the dropped neurons are effectively ignored. They don't contribute to the weight updates.
 - The gradients for the active neurons are scaled by a factor that accounts for the dropout probability

Derivative

- It is a way to measure the rate of change of a function with respect to an independent variable.
- It tells us how quickly a function is changing at a specific point.

$$y = f(x)$$

x: independent variable

y: dependent variable

Partial Derivative

- It measures the rate of change of a multivariable function
- It is done with respect to one of its variables, while holding all other variables constant.

$$f(x,y,z) = 2x+3y+4z$$

$$\frac{\partial f(x,y,z)}{\partial x} = 2$$

$$\frac{\partial f(x,y,z)}{\partial y} = 3$$

$$\frac{\partial f(x,y,z)}{\partial z} = 4$$

Gradient

- It is a vector with partial derivatives of a multivariate function.

For a function of two variables, i.e., $f(x_1, x_2)$

Gradient: $\nabla_x f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right]$

Chain Rule

- It is a formula for finding the derivative of a composite function.

$y = (3x + 3)^3$, making $t = 3x + 3$
then $y = t^3$

$$\frac{dt}{dx} = 3, \quad \frac{dy}{dt} = 3t^2$$

using the Chain Rule: $\frac{dy}{dx} = \frac{dy}{dt} \cdot \frac{dt}{dx}$

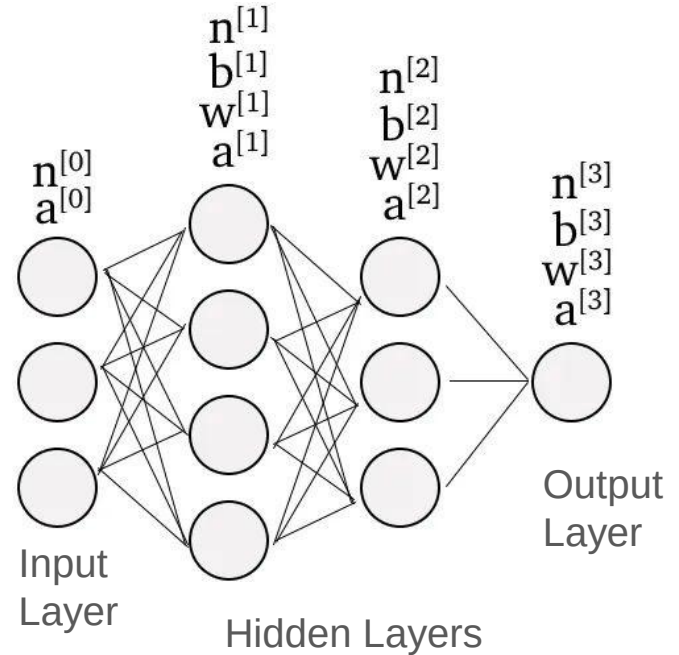
$$\begin{aligned} \therefore \frac{dy}{dx} &= (3t^2) \cdot (3) = 9t^2 \\ &= 9(3x + 3)^2 = 9(3)(x + 1)(3)(x + 1) \\ &= \underline{81(x + 1)^2} \end{aligned}$$

Backpropagation Algorithm

- It is used for efficient weight adjustments to minimize the error between predicted and actual outputs.
- It works by:
 - calculating the gradient of the loss function with respect to the network's weights.
 - Propagating error information backward through the network layers

A Simple Reference Neural Network

- n : number of neurons
- a : vector of activation function
- W : matrix of weights
- b : vector of biases



Formulas

- Weighted input to the neurons in layer l , $z^{[l]} = w^{[l]} \cdot a^{[l-1]} + b^{[l]}$
- Activation of layer l , $a^{[l]} = g^{[l]}(z^{[l]})$
- Say, one neuron's output y can be 0 or 1.

Then cross-entropy cost function is:

$$C = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

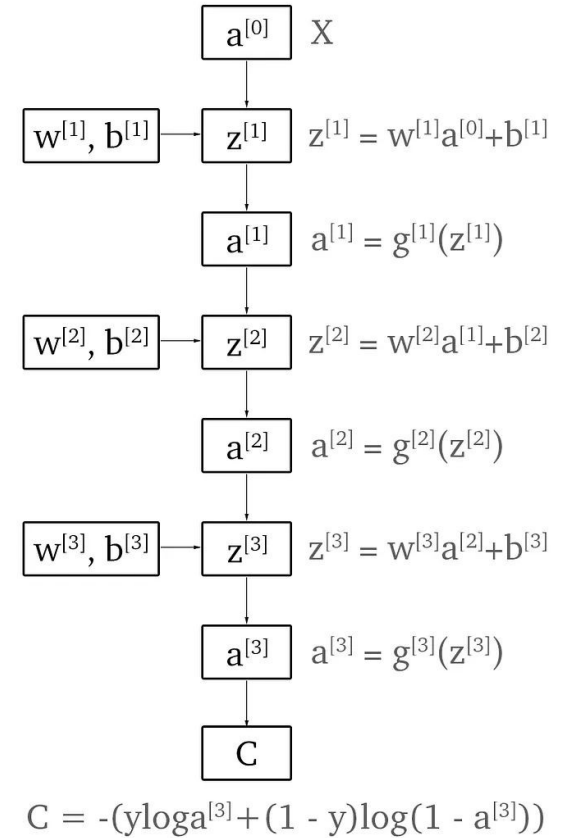
Update Rule

Alpha is the learning rate, which decides how big the updates will happen.

$$w^{[l]} = w^{[l]} - \alpha \frac{\partial C}{\partial w^{[l]}}$$

$$b^{[l]} = b^{[l]} - \alpha \frac{\partial C}{\partial b^{[l]}}$$

Forward Propagation Computation Graph



Partial Derivative of C

- Partial Derivative of C with respect to parameters of any layer, l

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial w^{[l]}}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial b^{[l]}}$$

Partial Derivative of C

- Partial Derivative of C with respect to parameters of Layer-3

$$\frac{\partial C}{\partial w^{[3]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial w^{[3]}}$$

$$\frac{\partial C}{\partial b^{[3]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial b^{[3]}}$$

Partial Derivative of C

- Partial Derivative of C with respect to parameters of Layer-2

$$\frac{\partial C}{\partial w^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

$$\frac{\partial C}{\partial b^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial b^{[2]}}$$

Partial Derivative of C

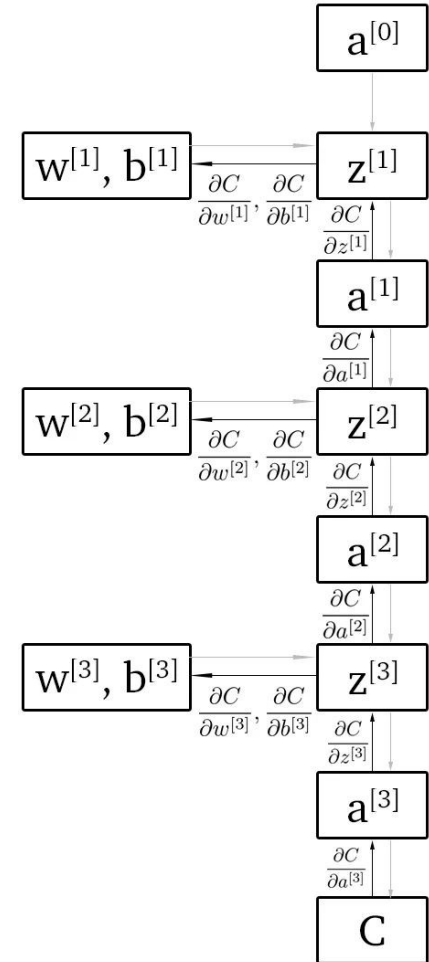
- Partial Derivative of C with respect to parameters of Layer-1

$$\frac{\partial C}{\partial w^{[1]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w^{[1]}}$$

$$\frac{\partial C}{\partial b^{[1]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial b^{[1]}}$$

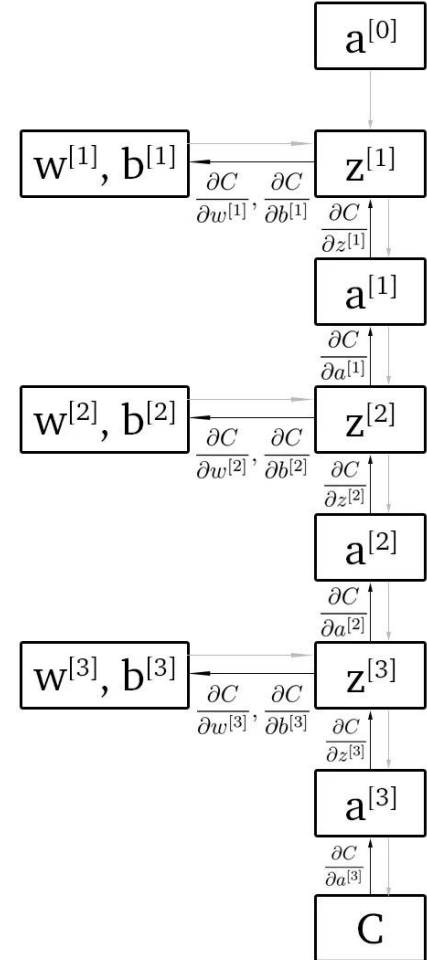
Backpropagation Computation Graph

- for calculating the partial derivatives of C with respect to $w[l]$, $b[l]$, we need to calculate $\frac{\partial C}{\partial z^{[L]}}$, $\frac{\partial C}{\partial z^{[l]}}$, $\frac{\partial z^{[l]}}{\partial w^{[l]}}$, $\frac{\partial z^{[l]}}{\partial b^{[l]}}$
- error information is propagated backward through the network layers for updating the weights



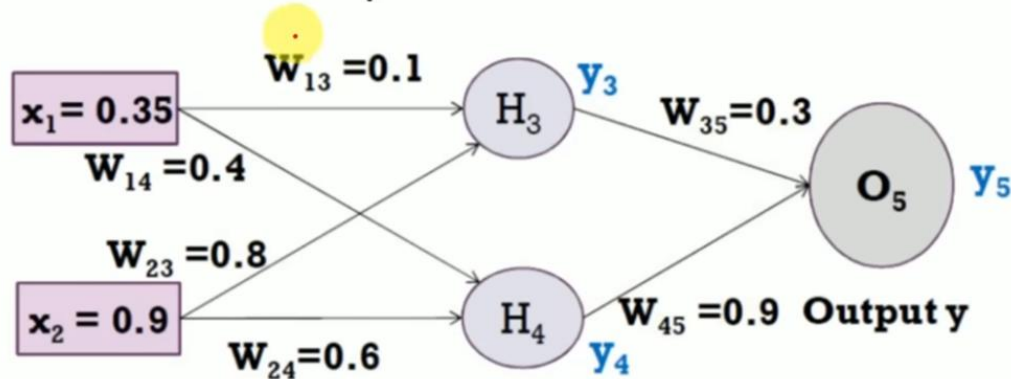
Backpropagation Computation Graph

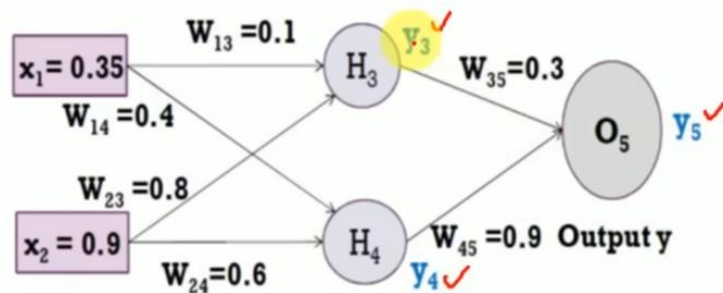
- for calculating the partial derivatives of C with respect to $w[l]$, $b[l]$, we need to calculate $\frac{\partial C}{\partial z^{[L]}}$, $\frac{\partial C}{\partial z^{[l]}}$, $\frac{\partial z^{[l]}}{\partial w^{[l]}}$, $\frac{\partial z^{[l]}}{\partial b^{[l]}}$
- error information is propagated backward through the network layers for updating the weights



Back Propagation Solved Example - 1

- Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 0.5 and learning rate is 1. Perform another forward pass.





- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_i (w_{ij} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$

$$\begin{aligned} a_1 &= (w_{13} * x_1) + (w_{23} * x_2) \\ &= (0.1 * 0.35) + (0.8 * 0.9) = 0.755 \\ y_3 &= f(a_1) = 1 / (1 + e^{-0.755}) = 0.68 \end{aligned}$$

$$\begin{aligned} a_2 &= (w_{14} * x_1) + (w_{24} * x_2) \\ &= (0.4 * 0.35) + (0.6 * 0.9) = 0.68 \\ y_4 &= f(a_2) = 1 / (1 + e^{-0.68}) = 0.6637 \end{aligned}$$

$$\begin{aligned} a_3 &= (w_{35} * y_3) + (w_{45} * y_4) \\ &= (0.3 * 0.68) + (0.9 * 0.6637) = 0.801 \\ y_5 &= f(a_3) = 1 / (1 + e^{-0.801}) = 0.69 \text{ (Network Output)} \end{aligned}$$

$$\text{Error} = Y_{\text{target}} - y_5 = -0.19$$

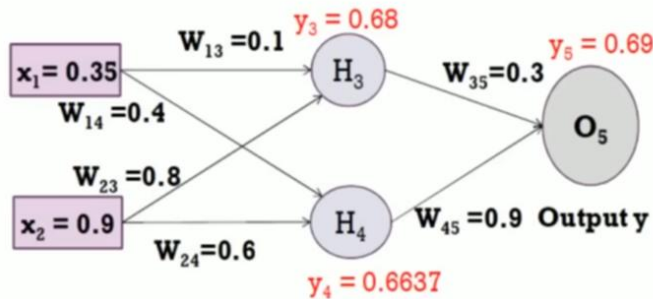
- Each weight changed by:

$$*w_{ij} \quad \Delta w_{ji} = \eta \delta_j o_i$$

$$\delta_j = o_j(1 - o_j)(t_j - o_j) \quad \text{if } j \text{ is an output unit}$$

$$\delta_j = o_j(1 - o_j) \sum_k \delta_k w_{kj} \quad \text{if } j \text{ is a hidden unit}$$

- where η is a constant called the learning rate
- t_j is the correct teacher output for unit j
- δ_j is the error measure for unit j



• Backward Pass: Compute δ_3 , δ_4 and δ_5 .

For output unit:

$$\delta_5 = y(1-y) (y_{\text{target}} - y) \\ = 0.69*(1-0.69)*(0.5-0.69) = -0.0406$$

For hidden unit:

$$\delta_3 = y_3(1-y_3) w_{35} * \delta_5 \\ = 0.68*(1 - 0.68)*(0.3 * -0.0406) = -0.00265$$

$$\delta_4 = y_4(1-y_4)w_{45} * \delta_5 \\ = 0.6637*(1 - 0.6637)* (0.9 * -0.0406) = -0.0082$$

Compute new weights

$$\Delta w_{ji} = \eta \delta_j o_i$$

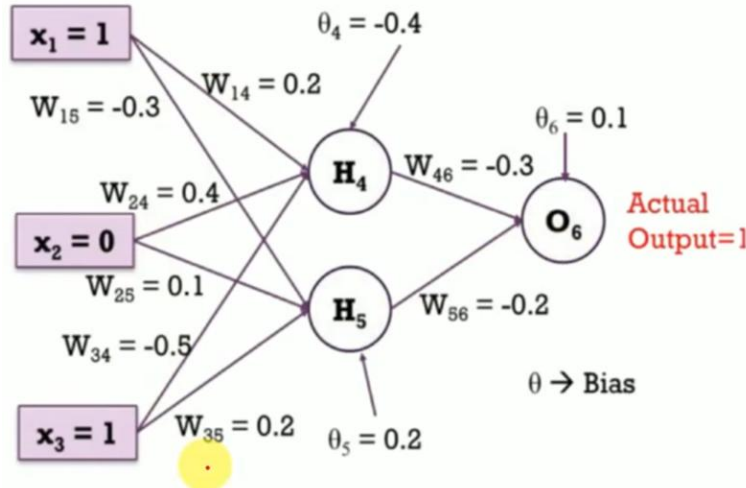
$$\Delta w_{45} = \eta \delta_5 y_4 = 1 * -0.0406 * 0.6637 = -0.0269 \\ w_{45}(\text{new}) = \Delta w_{45} + w_{45}(\text{old}) = -0.0269 + (0.9) = 0.8731$$

$$\Delta w_{14} = \eta \delta_4 x_1 = 1 * -0.0082 * 0.35 = -0.00287 \\ w_{14}(\text{new}) = \Delta w_{14} + w_{14}(\text{old}) = -0.00287 + 0.4 = 0.3971$$

- Similarly, update all other weights

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
1	3	0.1	-0.00265	0.35	1	0.0991
2	3	0.8	-0.00265	0.9	1	0.7976
1	4	0.4	-0.0082	0.35	1	0.3971
2	4	0.6	-0.0082	0.9	1	0.5926
3	5	0.3	-0.0406	0.68	1	0.2724
4	5	0.9	-0.0406	0.6637	1	0.8731

Back Propagation Solved Example - 2



Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 1 and learning rate is 0.9. Perform another forward pass.